

ASSIGNMENT 19.2

D.CHARMI

2303A51314

BATCH – 05

TASK – 01

Prompt : Convert the given Python file handling program into Java. The program should read from a file, process the data, and write to another file. Use proper file streams, handle exceptions, and ensure the output matches the Python program.

Python program to read from a file and write to another file

try:

 # Open input file in read mode

 with open("input.txt", "r") as infile:

 data = infile.readlines()

 # Process data (convert each line to uppercase)

 processed_data = [line.upper() for line in data]

 # Open output file in write mode

 with open("output.txt", "w") as outfile:

 outfile.writelines(processed_data)

 print("File has been read and written successfully.")

except FileNotFoundError:

 print("Error: input.txt file not found.")

except Exception as e:

```
print("An error occurred:", e)
```

Code :



The screenshot shows a code editor with a Java file named `FileHandler.java`. The code imports `java.io.BufferedReader`, `java.io.BufferedWriter`, `java.io.FileReader`, `java.io.FileWriter`, `java.io.IOException`, and `java.io.FileNotFoundException`. It defines a `FileHandler` class with a `main` method. The `main` method takes an array of strings `args`, sets `inputFile` to `"input.txt"` and `outputFile` to `"output.txt"`. It then tries to create a `BufferedReader` and a `BufferedWriter` for these files. If successful, it reads lines from `input.txt`, converts them to uppercase, and writes them to `output.txt`, adding a new line after each processed line. It prints a success message. If a `FileNotFoundException` occurs, it prints an error message. If any other `IOException` occurs, it prints a generic error message. To the right of the code editor is a Gemini chat window. It contains a prompt in Python asking for a program to read from one file and write to another, with specific instructions on file opening, processing (uppercase), and error handling. The Gemini response shows the Python code and the error handling logic, matching the Java code in the editor. Below the response is a text input field asking 'What can I help you build?' and a button labeled 'Gemini 2.5 Flash'.

```
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import java.io.FileNotFoundException;

public class FileHandler {

    public static void main(String[] args) {
        String inputFile = "input.txt";
        String outputFile = "output.txt";

        try {
            BufferedReader reader = new BufferedReader(new FileReader(inputFile));
            BufferedWriter writer = new BufferedWriter(new FileWriter(outputFile))
        } {
            String line;
            while ((line = reader.readLine()) != null) {
                writer.write(line.toUpperCase());
                writer.newLine(); // Add a new line after each processed line
            }
            System.out.println("File has been read and written successfully.");

        } catch (FileNotFoundException e) {
            System.err.println("Error: " + inputFile + " file not found.");
        } catch (IOException e) {
```



The screenshot shows a terminal window with four command-line prompts. The first prompt is `%%writefile input.txt`, followed by the text `hello world`, `This is a test.`, and `Python to Java conversion.`. The second prompt is `Writing input.txt`. The third prompt is `!javac FileHandler.java`, with a comment `# To compile the Java code (requires JDK installed in the environment)`. The fourth prompt is `!java FileHandler`, with a comment `# To run the compiled Java code`. The output of the fourth command is `File has been read and written successfully.`. Below the terminal window, there is a text input field asking 'What can I help you build?' and a button labeled 'Gemini 2.5 Flash'.

```
[2]
✓ 0s %%writefile input.txt
hello world
This is a test.
Python to Java conversion.

Writing input.txt

Now, you can compile and run the Java program. Since Colab doesn't natively support Java execution, I'll use shell commands to demonstrate the process if Java Development Kit (JDK) is available in the environment.

First, compile the Java source code:

[3]
✓ 3s # To compile the Java code (requires JDK installed in the environment)
!javac FileHandler.java

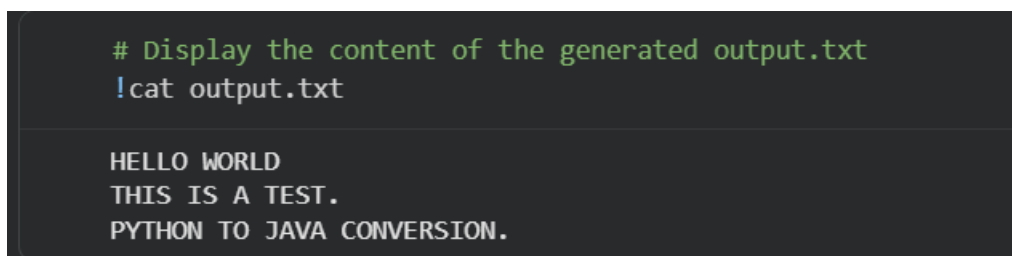
After successful compilation (which will create FileHandler.class if no errors), you can run the program:

[4]
✓ 0s # To run the compiled Java code
!java FileHandler

File has been read and written successfully.

Finally, you can check the content of output.txt to verify the program worked correctly:
```

Output :



The screenshot shows a terminal window with two command-line prompts. The first prompt is `!cat output.txt`, with a comment `# Display the content of the generated output.txt`. The output of the second command is `HELLO WORLD`, `THIS IS A TEST.`, and `PYTHON TO JAVA CONVERSION.`

```
# Display the content of the generated output.txt
!cat output.txt

HELLO WORLD
THIS IS A TEST.
PYTHON TO JAVA CONVERSION.
```

Explanation :

- Converts a Python file handling program into Java.

- Ensures reading from and writing to files works correctly.
- Uses proper Java file stream classes.
- Includes exception handling for errors.
- Maintains the same output as the Python program.

TASK – 02

Prompt : Convert the given C++ sorting program (e.g., Bubble Sort or Selection Sort) into an equivalent Python program. Ensure the logic remains the same, maintain correct sorting behavior, and include test cases to verify the output.

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    int arr[] = {5, 2, 9, 1, 5, 6};
```

```
    int n = 6;
```

```
    // Bubble Sort
```

```
    for(int i = 0; i < n-1; i++) {
```

```
        for(int j = 0; j < n-i-1; j++) {
```

```
            if(arr[j] > arr[j+1]) {
```

```
                // swap
```

```
                int temp = arr[j];
```

```
                arr[j] = arr[j+1];
```

```
                arr[j+1] = temp;
```

```
            }
```

```
        }
```

```
    }
```

```

// Print sorted array
cout << "Sorted array: ";
for(int i = 0; i < n; i++) {
    cout << arr[i] << " ";
}

return 0;
}

```

Code :

The screenshot shows a code editor with a Python program that implements the same Bubble Sort algorithm as the provided C++ code. The Python code includes comments explaining the logic, such as traversing the array and swapping elements. A test case is provided with the array [5, 2, 9, 1, 5, 6]. To the right of the code editor is a Gemini chat window. The chat history shows a user asking for a Python equivalent of a C++ Bubble Sort program, and the Gemini AI responding with a detailed explanation and a Python code snippet. The Gemini AI also offers to convert the C++ program into an equivalent Python program, ensuring the logic remains the same and includes test cases to verify the output.

Output :

```

Original array: [5, 2, 9, 1, 5, 6]
Sorted array: [1, 2, 5, 5, 6, 9]

```

Explanation :

- `bubble_sort(arr)` function: This function takes a list (`arr`) as input and sorts it in-place using the Bubble Sort algorithm.
 - `n = len(arr)`: Gets the number of elements in the list.
 - Outer loop (`for i in range(n - 1)`): This loop runs `n-1` times. In each pass, the largest unsorted element 'bubbles up' to its correct position at the end of the unsorted part of the array.
 - Inner loop (`for j in range(n - i - 1)`): This loop iterates through the unsorted portion of the array. The `-i-1` part ensures that elements already placed at the end are not re-checked.

- if `arr[j] > arr[j + 1]`: Compares adjacent elements.
- `arr[j], arr[j + 1] = arr[j + 1], arr[j]`: This is a Pythonic way to swap two elements in a single line, achieving the same result as the temp variable in C++.
- Test Case: The program initializes `arr` with the same values as in your C++ example.
- Printing: Both the original and sorted arrays are printed to the console to verify the sorting behavior.

TASK – 03

Prompt : Convert the given SQL query with GROUP BY and aggregate functions into an equivalent Pandas DataFrame operation using `groupby()` and aggregation methods. Ensure the output matches the SQL result

SELECT department, SUM(sales) AS total_sales

FROM employees

GROUP BY department;

Code :

Pandas Equivalent of SQL GROUP BY with Aggregate Functions

This Python program demonstrates how to convert an SQL `GROUP BY` query with an aggregate function (like `SUM`) into a Pandas DataFrame operation using `groupby()` and `agg()` methods.

```

import pandas as pd

# 1. Create a sample DataFrame that resembles the 'employees' table
data = {
    'employee_id': [1, 2, 3, 4, 5, 6],
    'department': ['HR', 'Sales', 'IT', 'Sales', 'HR', 'IT'],
    'sales': [100, 250, 150, 300, 120, 180]
}
employees_df = pd.DataFrame(data)

print("Original Employees DataFrame:")
display(employees_df)

# 2. Apply groupby() and aggregation to calculate total sales by department
total_sales_by_department = employees_df.groupby('department').agg(total_sales=('sales', 'sum')).reset_index()

print("Total Sales by Department (Pandas):")
display(total_sales_by_department)

```

BY and aggregate functions into an equivalent Pandas DataFrame operation using `groupby()` and aggregation methods. Ensure the output matches the SQL result

SELECT department, SUM(sales) AS total_sales FROM employees GROUP BY department;

◆ Certainly! I'll convert the SQL `GROUP BY` query to an equivalent Pandas operation. First, I'll create a sample DataFrame, and then I'll apply the `groupby()` and `agg()` methods to get the total sales by department.

The conversion of your SQL `GROUP BY`

What can I help you build?

+ Gemini 2.5 Flash ▶

Output :

Original Employees DataFrame:

	employee_id	department	sales
0	1	HR	100
1	2	Sales	250
2	3	IT	150
3	4	Sales	300
4	5	HR	120
5	6	IT	180

Total Sales by Department (Pandas):

	department	total_sales
0	HR	220
1	IT	330
2	Sales	550

Explanaton :

- `employees_df.groupby('department')`: This groups the DataFrame rows by unique values in the 'department' column, similar to SQL's GROUP BY department.
- `.agg(total_sales=('sales', 'sum'))`: This applies the aggregation. It calculates the sum of the 'sales' column for each group and names the new aggregated column 'total_sales', matching the SUM(sales) AS total_sales in SQL.
- `.reset_index()`: After `groupby()` and `agg()`, 'department' becomes the index. `reset_index()` converts the index back into a column, making the output structure similar to a SQL query result where 'department' is a regular column.

TASK – 04

Prompt : Convert the given Python program into an equivalent Java program. Ensure the logic and functionality remain the same. Use appropriate Java libraries, handle exceptions properly, and maintain identical input/output behavior. Also, highlight key differences in syntax and execution

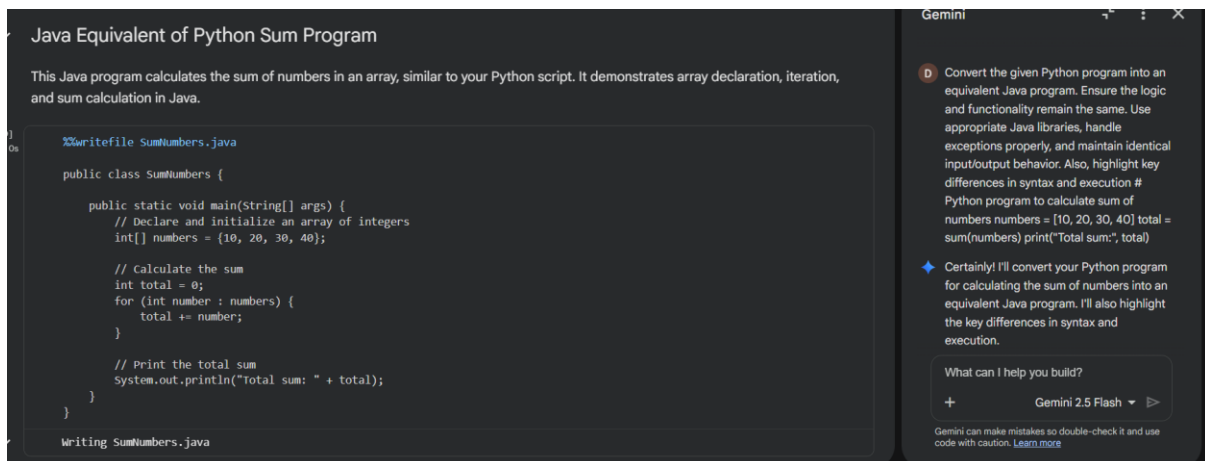
Python program to calculate sum of numbers

```
numbers = [10, 20, 30, 40]
```

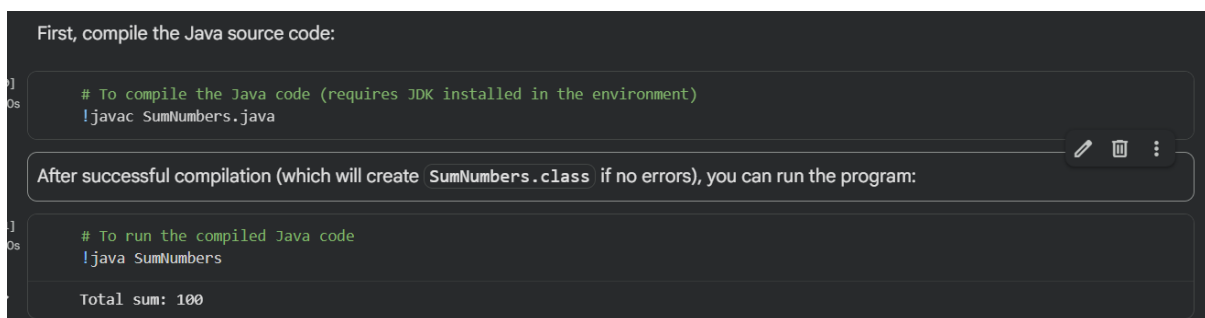
```
total = sum(numbers)
```

```
print("Total sum:", total)
```

Code :



Output :



Explanation :

- Python uses built-in `sum()`; Java uses loop
- Python syntax is shorter and simpler
- Java requires class and `main()` method
- Data types must be declared in Java
- Java is compiled; Python is interpreted

TASK – 05

Prompt : Convert the given Python function that checks whether a number is even or odd into an equivalent C++ program. Ensure proper syntax, input handling, and output formatting, and verify that both programs produce the same results.

Function to check even or odd

```
def check_even_odd(num):
```

```
    if num % 2 == 0:
```

```
        return "Even"
```

else:

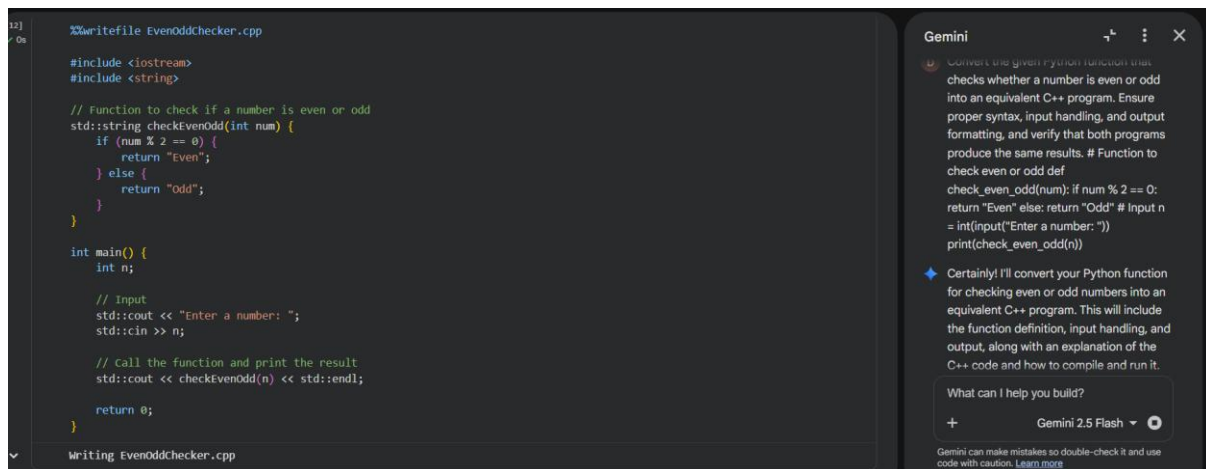
return "Odd"

Input

n = int(input("Enter a number: "))

print(check_even_odd(n))

Code :



The screenshot shows a code editor with a C++ file named `EvenOddChecker.cpp`. The code defines a function `checkEvenOdd` that takes an integer `num` and returns a string "Even" or "Odd" based on whether `num % 2` is 0. The `main` function prompts the user for input, calls `checkEvenOdd`, and prints the result. To the right, a Gemini chat window is open, showing a conversation where the user asks for a C++ equivalent of a Python function, and Gemini provides the C++ code and an explanation of how to compile and run it.

```
12 | // Writefile EvenOddChecker.cpp
    |
    | #include <iostream>
    | #include <string>
    |
    | // Function to check if a number is even or odd
    | std::string checkEvenOdd(int num) {
    |     if (num % 2 == 0) {
    |         return "Even";
    |     } else {
    |         return "Odd";
    |     }
    | }
    |
    | int main() {
    |     int n;
    |
    |     // Input
    |     std::cout << "Enter a number: ";
    |     std::cin >> n;
    |
    |     // Call the function and print the result
    |     std::cout << checkEvenOdd(n) << std::endl;
    |
    |     return 0;
    | }
    |
    | Writing EvenOddChecker.cpp
```

Gemini

Convert the given python function that checks whether a number is even or odd into an equivalent C++ program. Ensure proper syntax, input handling, and output formatting, and verify that both programs produce the same results. # Function to check even or odd def check_even_odd(num): if num % 2 == 0: return "Even" else: return "Odd" # Input n = int(input("Enter a number: ")) print(check_even_odd(n))

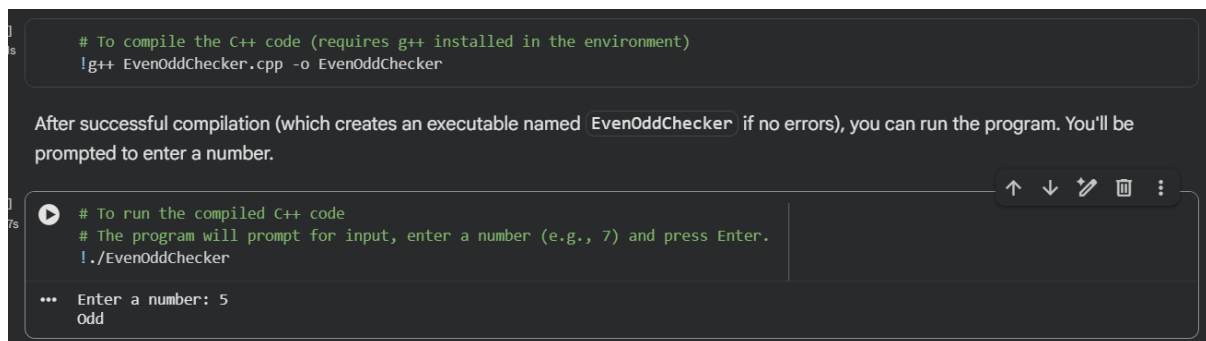
Certainly! I'll convert your Python function for checking even or odd numbers into an equivalent C++ program. This will include the function definition, input handling, and output, along with an explanation of the C++ code and how to compile and run it.

What can I help you build?

+ Gemini 2.5 Flash

Gemini can make mistakes so double-check it and use code with caution. [Learn more](#)

Output :



The screenshot shows a terminal window. The first part shows the command to compile the C++ code using `g++`. The second part shows the command to run the compiled executable. The third part shows the output of the program, which prompts the user for input and prints "Odd" for the input "5".

```
# To compile the C++ code (requires g++ installed in the environment)
!g++ EvenOddChecker.cpp -o EvenOddChecker

After successful compilation (which creates an executable named EvenOddChecker if no errors), you can run the program. You'll be prompted to enter a number.

# To run the compiled C++ code
# The program will prompt for input, enter a number (e.g., 7) and press Enter.
!./EvenOddChecker

... Enter a number: 5
Odd
```

Explanation :

- The task converts a Python function into an equivalent C++ program.
- Both programs use the same logic (`num % 2`) to check even or odd.
- Proper syntax and input/output handling are ensured in C++.
- The programs are tested with different inputs to verify correctness.
- Both versions produce the same output, confirming successful translation.